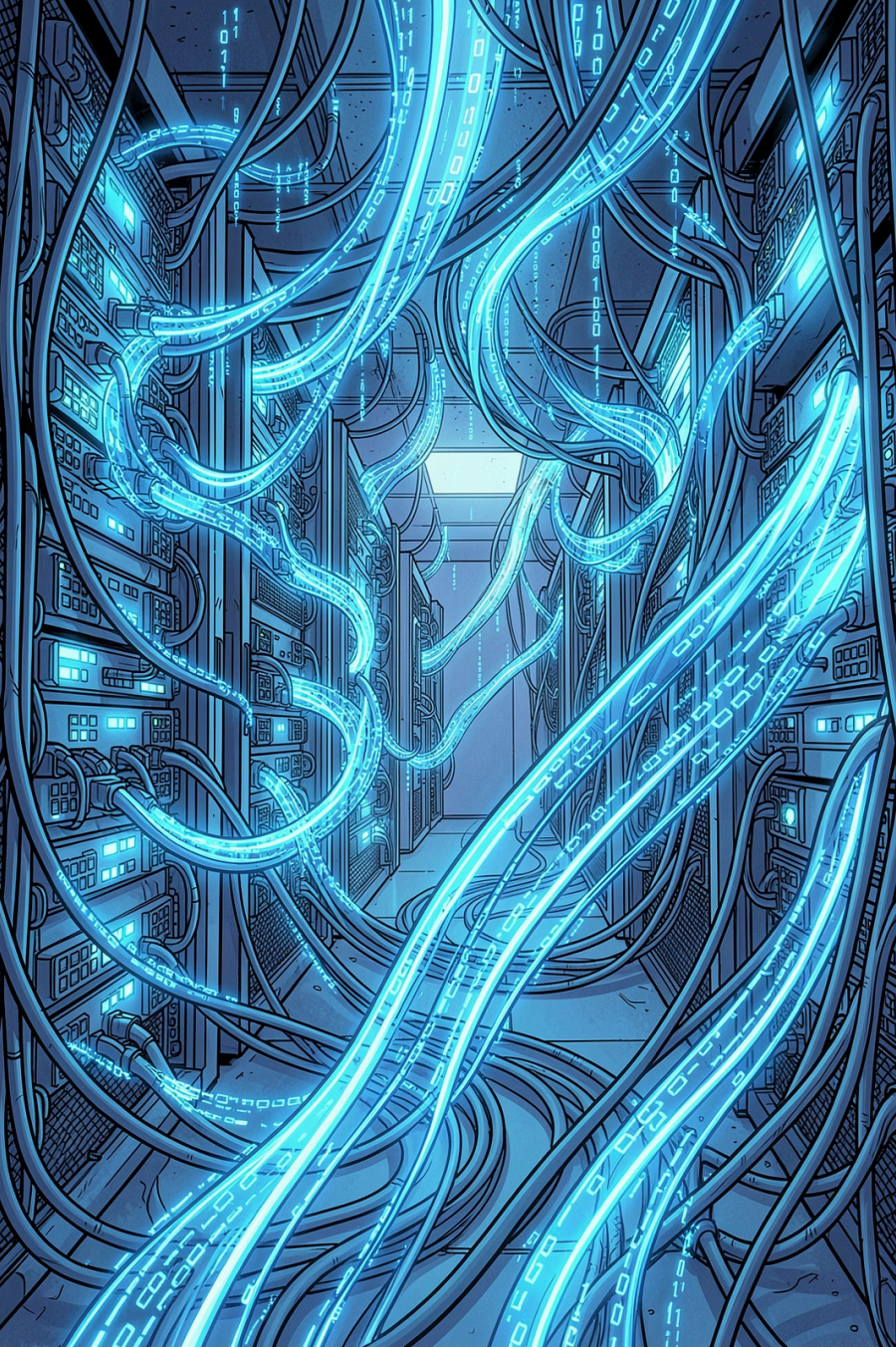




SQL Server: Automatización y Control Estratégico

Dominando **Vistas y Triggers** para optimizar tu arquitectura de datos

Vistas — El Arte de la Abstracción



¿Qué son?

Tablas virtuales que encapsulan consultas complejas bajo un nombre simple.

¿Para qué sirven?

Seguridad (ocultan columnas), abstracción de lógica y legibilidad del código.

¿Cómo funcionan?

Se ejecutan como un `SELECT` cada vez que se invocan, reflejando datos en tiempo real.

Aplicación de Vistas en Código

Escenario

Ocultar columnas sensibles (salario, DNI) de la tabla `Empleados`, exponiendo solo datos seguros a usuarios estándar.

```
CREATE VIEW VistaEmpleados AS  
SELECT Nombre, Puesto, Departamento  
FROM Empleados  
WHERE Activo = 1;
```

 Los usuarios consultan `VistaEmpleados` sin acceso directo a la tabla original.

Beneficios inmediatos

- Las columnas sensibles quedan inaccesibles
- Solo empleados activos son visibles
- Si la tabla cambia, la vista se adapta con `ALTER VIEW`
- Reducción de errores en consultas repetitivas

CONSIDERACIONES CRÍTICAS

Lo Que Debes Tener en Cuenta con las Vistas

Sin almacenamiento físico

Son reflejos dinámicos de la tabla base. No ocupan espacio, pero dependen de la tabla original.

Impacto en rendimiento

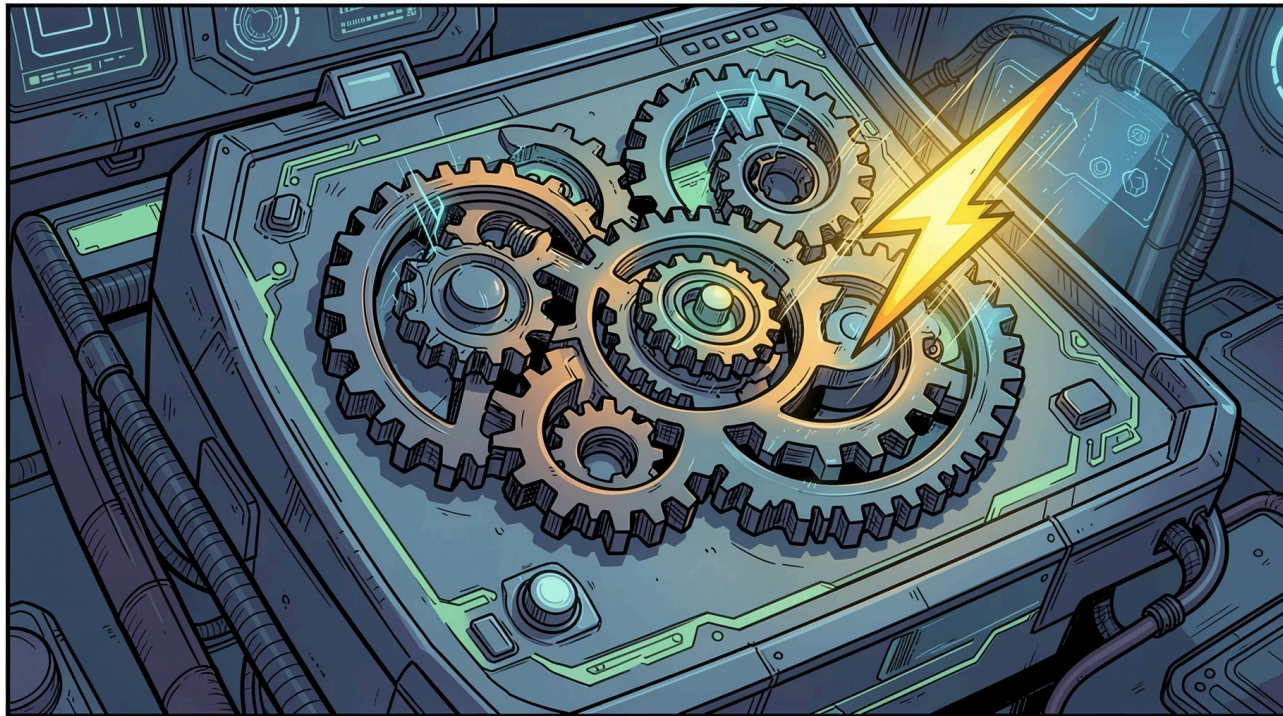
Consultas complejas sobre vistas anidadas pueden degradar la velocidad. Analiza el plan de ejecución.

Mantenimiento seguro

Usa siempre `ALTER VIEW` en lugar de `DROP + CREATE` para preservar permisos asignados.



Triggers — La Inteligencia Automática



¿Qué son?

Procedimientos almacenados que se **disparan automáticamente** ante eventos específicos en la base de datos.

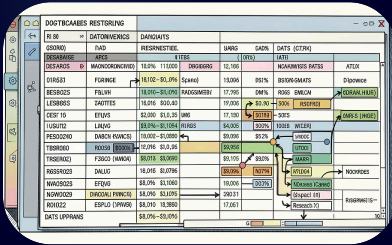
¿Para qué sirven?

Auditoría de cambios, integridad referencial avanzada y automatización de reglas de negocio.

¿Cómo funcionan?

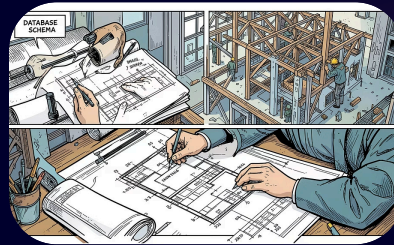
Se ejecutan tras eventos INSERT, UPDATE, DELETE (DML) o cambios de esquema (DDL).

Tipos de Disparadores



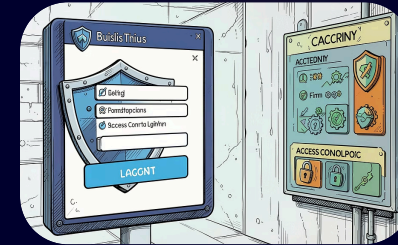
DML Triggers

Responden a cambios en **datos**: INSERT, UPDATE, DELETE sobre tablas o vistas.



DDL Triggers

Reaccionan a cambios en la **estructura**: CREATE, ALTER, DROP de objetos.



Logon Triggers

Se activan cuando un usuario **inicia sesión** en el servidor. Ideales para auditoría de acceso.

Aplicación de Triggers: Auditoría Automática

Escenario

Registrar automáticamente cada modificación en una tabla de auditoría sin intervención del usuario.

```
CREATE TRIGGER AuditoriaCambios
ON Ventas
AFTER UPDATE
AS
BEGIN
    INSERT INTO LogAuditoria
    (Tabla, Usuario, Fecha)
    VALUES ('Ventas',
            SYSTEM_USER,
            GETDATE());
END;
```

Claves del ejemplo

- **AFTER UPDATE**: se ejecuta tras confirmar el cambio
- **SYSTEM_USER**: captura quién hizo el cambio
- **GETDATE()**: registra el momento exacto
- También existe **INSTEAD OF** para interceptar la acción

Qué Tener en Cuenta con los Triggers

1

Precaución con el rendimiento

El uso excesivo degrada severamente el sistema. Cada trigger añade latencia a cada operación.

2

Mantén la simplicidad

Deben ser **cortos, rápidos y enfocados** en una sola tarea. Evita lógica compleja dentro de ellos.

3

Controla la recursividad

Un trigger que modifica la misma tabla puede llamarse a sí mismo. Usa `RECURSIVE_TRIGGERS OFF`.

4

Documenta siempre

Los triggers son invisibles para el desarrollador. Añade comentarios explicando cuándo y por qué se disparan.



El Guardián de Datos

Objetivo del reto

Crea un **trigger** que impida borrar registros de la tabla **Ventas** si el usuario no tiene rol **admin**, y una **vista** que solo muestre las ventas del mes actual.

⚠️ Pista: usa **SYSTEM_USER** para validar el rol y **MONTH(GETDATE())** para filtrar el mes.

Requisitos técnicos

- Trigger tipo **INSTEAD OF DELETE** con validación de rol
- Mensaje de error claro si el usuario no es admin
- Vista con filtro dinámico por mes actual
- Prueba con usuario estándar y admin

Entregable esperado

Script SQL completo con ambos objetos y comentarios explicativos.

Conclusiones

Vistas

Tu herramienta para **simplificar el acceso**, proteger información sensible y abstraer la complejidad de las consultas.

Triggers

Tu motor de **reglas de negocio automatizado**. Garantizan auditoría, integridad y consistencia sin depender del código de aplicación.

El Futuro

Ambos pilares son **esenciales** para una base de datos robusta, eficiente y mantenible a largo plazo.



Recuerda: Usa vistas para simplificar y triggers para automatizar, pero siempre con moderación y documentación clara.

